Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа № 4

«Операционные системы реального времени»

по дисциплине

«Проектирование вычислительных систем»

Вариант № 2: «Компас»

Выполнили: Мироненко Артем Дмитриевич,
Дмитрий Амельченко
Группа: P3431
Преподаватель: Пинкевич Василий Юрьевич

Санкт-Петербург,

2026

Оглавление

1. Цели работы и описание задания 3

2. Аппаратная платформа и подключение 4

3. Магнитометр и расчёт направления 5

4. OLED и организация обмена I²C 6

5. Протокол и клиентская программа 7

6. Задачи FreeRTOS и синхронизация 8

7. Методика и результаты программной проверки 9

8. Сборка, загрузка и воспроизводимость 10

9. Сравнение реализаций и ограничения 11

10.1. Исходные коды: главный цикл 12

10.2. Исходные коды: прикладная логика 13

10.3. Исходные коды: драйвер устройств 14

11. Проверка на стенде, выводы и источники 15

Дата программной проверки: 02.11.2026. Аппаратная проверка вынесена в отдельный протокол. Оформление титульного листа, исполнители и преподаватель сохранены по отчёту первой лабораторной.

1. Цели работы и описание задания

Цели

Изучить FreeRTOS и перенос приложения компаса на вытесняющий планировщик. Сохранить функции ЛР3, реализовать минимум две задачи и обработчик прерывания, обеспечить стабильный вывод не менее 20 кадров/с со счётчиком 00…99.

Функции стенда

Постоянно выводить компоненты магнитного поля X, Y, Z и стрелку направления поля при горизонтальном расположении платы. Применять присланные коэффициенты каждой оси 0…200%, по умолчанию 100%. Коэффициенты сохраняются в ОЗУ до следующего изменения или сброса.

Функции клиента

COM-порт задаётся аргументом. После открытия немедленно передаётся PING, ожидается PONG. Интерфейс показывает компоненты, направление и состояния, позволяет кнопками изменять коэффициенты. Все переданные и принятые байты сохраняются в JSONL с временем UTC. Построчный ввод команд пользователем не используется.

Источник

SDK-1.1M_26101101.pdf, разделы 5.3 и 5.4. В этой редакции ЛР3 — компас с OLED, ЛР4 — тот же компас на FreeRTOS. Бонусная клавиатура не включена в реализацию. Аппаратные измерения не выполнялись.

2. Аппаратная платформа и подключение

Аппаратные подключения

Стенд SDK-1.1MC.407 либо SDK-1.1MC.427. Для сборки используются соответствующие определения STM32F407xx и STM32F427xx. Встроенный FTDI подключается через разъём DBG USB: канал JTAG используется OpenOCD, канал UART — терминалом или клиентом.

Тактирование

HSI 16 МГц, PLLM=16, PLLN=336, PLLP=4: SYSCLK=84 МГц. AHB=84 МГц, APB1=42 МГц, APB2=84 МГц. При делителе APB1=2 частота TIM6 равна 84 МГц. Flash: 2 wait states при питании 3,3 В. Независимость от кварца упрощает перенос между двумя модулями стенда.

Временная база

HAL_GetTick получает миллисекунды от TIM6: PSC=83, ARR=999. Прерывание TIM6_DAC подключено в таблице векторов. Таймер используется и до запуска приложения, поэтому тайм-ауты HAL работают при инициализации PLL и I²C.

I²C

I²C1, PB8/SCL и PB9/SDA, AF4, open drain, внешние подтяжки стенда. Частота 400 кГц. OLED: 7-битный адрес 0x3C, в HAL передаётся 0x78. PCA9538 сброса OLED: 7-битный 0x70, адресный байт 0xE0; используется P0.

Датчик

LSM9DS1 определяется значением WHO_AM_I=0x3D по адресу 0x1C или 0x1E. Для совместимости с другой ревизией поддержан LSM9DS0, WHO_AM_I=0x49 по 0x1D/0x1E. Настройка производится только после распознавания. USART6 PC6/PC7 AF8 работает на 115200 8N1.

3. Магнитометр и расчёт направления

Режим измерений

LSM9DS1: ultra-high performance по XY/Z, ODR 80 Гц, ±4 gauss, непрерывный режим, BDU. LSM9DS0: high resolution, 100 Гц, ±2 gauss, непрерывный режим, BDU. Читается флаг ZYXDA; шесть байтов XYZ считываются одной транзакцией с auto-increment.

Единицы

Сырые данные собираются как int16_t little-endian. Для DS1 используется 14,615 nT/LSB (приближение чувствительности 6842 LSB/gauss), для DS0 — 8 nT/LSB. В вычислении участвует int64_t, чтобы произведение отрицательного raw, чувствительности и масштаба не переполняло 32 бита.

Масштабирование

Bi[nT] = rawi × sensitivity[nT×1000/LSB] × scalei[%] / 100000. На ПК и OLED компоненты показаны в μT, для чего nT делятся на 1000. Набор масштабов применяется атомарно; значения вне 0…200% отклоняются.

Указатель

heading = atan2(By, Bx) × 180/π, затем нормализация в [0;360). В протоколе угол хранится в сотых градуса. Нулевой горизонтальный вектор даёт heading=-1. На указателе X+ — вверх, Y+ — вправо; dx=L sin(heading), dy=−L cos(heading).

Физические ограничения

При ошибке чтения или отсутствии новых измерений более 500 мс данные помечаются неактуальными, стрелка скрывается. Для географического направления стенд держат горизонтально и экспериментально сверяют оси. Наклон, намагниченность корпуса и близкий магнит изменяют измеряемое локальное поле. Проверка ориентации ещё предстоит.

4. OLED и организация обмена I²C

Видеобуфер

Разрешение OLED 128×64, framebuffer 1024 байта. Байт описывает вертикальную группу 8 пикселов: offset=x+128×(y/8), bit=y%8. Реализованы очистка, пиксел, линия Брезенхэма, текст компактным 3×5 шрифтом и стрелка с наконечником.

Сброс и инициализация

Перед инициализацией читаются output/config PCA9538. Изменяется только P0, остальные сигналы сохраняются. Формируется высокий уровень, низкий импульс 1 мс и восстановление с паузой 2 мс. SSD1306 переводится в горизонтальную адресацию, задаются MUX=63, COM/SEG remap и charge pump, затем display on.

Передача кадра

Сначала командами 0x21/0x22 выбирается область 0…127 и страницы 0…7. Затем отправляются 32 транзакции по 32 байта с управляющим байтом 0x40. Команды используют управляющий байт 0x00. Короткие пакеты ограничивают продолжительность отдельной блокирующей операции I²C.

Оценка скорости

При 400 кГц 32×34×9 бит /400000 ≈24,48 мс на полезные транзакции кадра, плюс START/STOP, команды, датчик и работа CPU. Целевой период 40 мс соответствует 25 FPS и оставляет расчётный запас. Реальная частота ≥20 FPS требует проверки на устройстве.

Единый владелец

Транзакции выполняются последовательно одним владельцем шины. В ЛР3 им является главный цикл; в ЛР4 — display task. UART ISR не обращается к I²C. Если OLED не отвечает, UART продолжает работать, повтор инициализации ограничен одним разом в секунду.

5. Протокол и клиентская программа

Команды

Протокол v1 — ASCII с LF/CRLF. PING → PONG COMPASS 1; GET → DATA; SCALE X Y Z → OK SCALE либо OK SCALE QUEUED. Допустимы только три целых коэффициента 0…200. Незнакомая, слишком длинная или некорректная команда даёт ERR COMMAND.

Телеметрия

DATA tick Bx By Bz heading sx sy sz sensor_ok oled_ok sensor_id frames frame_ms. Первые три компоненты — nT; угол — сотые градуса либо −1; tick — мс. Последние поля позволяют наблюдать состояние аппаратуры, число переданных кадров и интервал между завершениями кадров. Автоматическая передача — около 10 Гц.

Интерфейс

Клиент Python использует curses TUI. X/Y/Z и Left/Right выбирают ось, +/- и Up/Down меняют масштаб на 1%, PgUp/PgDn — на 10%, Enter отправляет настройки, R восстанавливает 100%, Q закрывает порт. Показаны редактируемые и фактически применённые значения.

Журнал

Каждый UART TX/RX-фрагмент записывается в JSONL: time в UTC, direction, hex, декодированный text, признак demo. Сохранение исходных hex-байтов позволяет восстановить поток при дроблении строк. По PONG подтверждается связь, отсутствие DATA более 1 с отмечается STALE.

Работа без платы

Режим --demo явно обозначен, генерирует синтетическое поле и ответы. Он проверяет интерфейс и логирование, но не датчик. Клиент также испытан через настоящий последовательный API pyserial на паре PTY с фрагментированными строками.

6. Задачи FreeRTOS и синхронизация

Исполнение

display task: приоритет 2, стек 1024 слова; единственный владелец I²C и структуры Compass, период vTaskDelayUntil 40 мс. communications task: приоритет 3, стек 768 слов; UART и парсер, не более 32 входных байтов за проход, затем vTaskDelay 1 мс.

Обмен данными

Две очереди FreeRTOS длины 1: scale_queue хранит последний полный набор коэффициентов; status_queue — последний полный снимок Compass. xQueueOverwrite заменяет устаревшее значение. Доступа двух задач к framebuffer нет. QUEUED подтверждает доставку в очередь; применение подтверждается DATA.

Прерывания

Порт GCC ARM_CM4F использует SVC, PendSV и SysTick. Частота tick 1 кГц, вытесняющая многозадачность. TIM6 отдельно формирует HAL tick. USART6 priority 6, TIM6 priority 15; max syscall priority 5. USART6 не вызывает API ОС из ISR и общается через SPSC.

Ошибки

Нет ответа магнитометра — sensor_ok=0, heading=-1, данные признаются неактуальными. Нет OLED — oled_ok=0. Частота кадров не считается подтверждённой при ошибках. Ошибки не создают неограниченного ожидания: операции I²C ограничены тайм-аутом 2 мс.

Ресурсы

Динамическое выделение FreeRTOS — heap_4 40 КиБ. Newlib получает контекст каждой задачи, её malloc защищён приостановкой планировщика. Hooks malloc failed и stack overflow останавливают исполнение для отладки. Необходимый запас стеков проверяется на стенде.

7. Методика и результаты программной проверки

Проверки выполнены без подключения к стенду. Тесты запускают реальные C-модули, а аппаратный HAL заменяется имитатором только в тестах драйверов. Компиляция прошивок выполняется настоящим ARM GCC. Прохождение этих проверок не подтверждает работу физических выводов.

Проверка	Результат и граница доказательства
ARM-сборка	HAL для STM32F407 и STM32F427; ELF, HEX, BIN. Для ЛР2 дополнительно REGISTER для обеих моделей.
C-логика	Clang -Wall -Wextra -Werror, ASan/UBSan. Ошибок в выполненных сценариях не выявлено.
I²C/OLED	Имитатор проверяет адреса, WHO_AM_I DS1/DS0, reset с сохранением P1…P7, 1024 байта кадра, ошибки и повтор.
Клиент ПК	3 Python-теста: протокол, demo и логи, pyserial через PTY с разбиением строк; все прошли.
Структура ELF	Проверены таблица векторов, адрес стека, heap; в ЛР4 — SVC/PendSV/SysTick и символы FreeRTOS.
OpenOCD	Разбор конфигураций .407/.427 с -c shutdown: успешен; к адаптеру не подключался.

Фактические логи: artifacts/tests.log, build_*.log, openocd_*.log. SHA256 всех готовых образов: artifacts/sha256.json. Повторить комплект: ../verify.sh. Испытания на плате: ПРОВЕРКА_НА_ПЛАТЕ.md.

8. Сборка, загрузка и воспроизводимость

Инструменты

Arm GNU Toolchain 14.2.Rel1 (AArch64 macOS), OpenOCD 0.12.0, Clang с AddressSanitizer и UndefinedBehaviorSanitizer; клиент Python 3.10 с pyserial 3.5. Версии исходников HAL, CMSIS и FreeRTOS закреплены commit SHA в Drivers/versions.json.

Сборка

Импортировать Compass_RTOS_LR4_V2 в STM32CubeIDE как Existing Project либо выполнить make -C Compass_RTOS_LR4_V2. Для другой модели задать MCU=STM32F407xx. Внешний Makefile автоматически ищет компилятор IDE, затем локальный tools/toolchain. STM32CubeMX и загрузка пакета HAL не требуются.

Прошивка

Из папки лабораторной выполнить ./flash.sh; для .407 — MCU=STM32F407xx ./flash.sh. Запускается сборка, OpenOCD через FTDI/JTAG, program verify reset exit. Конфигурации различают модели, в .427 добавлен boundary-scan TAP. Перед реальной записью сверить маркировку.

Отладка

./flash.sh --debug запускает GDB-сервер. Подключиться к localhost:3333, reset halt, load, break main, continue. Проверить достижение main, работу TIM6 и обработчиков. При остановке отладчиком исполнение приложения прекращается, поэтому FPS измеряется без остановок.

Запуск клиента

../tools/venv/bin/python client/client.py PORT --log uart.jsonl. Без платы — --demo с отдельным журналом. В журнале demo=true отличает синтетические данные. Установка на другом ПК описана в README.md.

9. Сравнение реализаций и ограничения

Без ОС

ЛР3: один управляющий цикл, последовательное владение I²C, предсказуемый порядок обработки и меньше RAM. Для отзывчивости нужно вручную разбивать передачу экрана и регулярно вызывать UART service. Рост числа функций усложняет расписание.

С ОС

ЛР4: две задачи разделяют обмен с ПК и работу с устройствами, а вытеснение обеспечивает обслуживание UART независимо от текущего блока OLED. Очереди дают целостные снимки. Требуются отдельные стеки, heap, правильные ISR-приоритеты и анализ загрузки.

Частота кадров

Целевой период обеих версий — 40 мс. В ЛР4 vTaskDelayUntil не добавляет длительность работы к каждому периоду. Если работа превысит бюджет, планировщик не может гарантировать FPS сам по себе. Счётчик frames и frame_ms позволяют проверить фактическое поведение; итоговые замеры пока отсутствуют.

Выбор синхронизации

Один владелец I²C исключает перекрытие транзакций без мьютекса. Очереди длины 1 подходят для последнего измерения и последнего набора коэффициентов; старые значения не нужно воспроизводить. Поток UART сохраняет порядок в обычном FIFO.

Границы

Не реализованы бонусная клавиатура, коррекция наклона и полная магнитная калибровка. Включены распознавание DS1/DS0 и выявление устаревших данных. Положение осей, географическое направление и минимальные FPS нужно подтвердить на стенде.

10.1. Исходные коды: главный цикл

Compass_RTOS_LR4_V2/Core/Src/main.c

```
#include "board.h"
#include "serial.h"
#include "devices.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
static QueueHandle_t scale_queue,status_queue;
void vApplicationMallocFailedHook(void) { fatal(); }
void vApplicationStackOverflowHook(TaskHandle_t task,char *name) { (void)task; (void)name; fatal(); }
/* This task alone owns I2C, OLED framebuffer, sensor and authoritative scales. */
static void display_task(void *arg) {
    (void)arg; Compass c; compass_init(&c); devices_init();
    TickType_t wake=xTaskGetTickCount(); uint32_t previous=HAL_GetTick();
    for(;;) {
        unsigned scale[3];
        if(xQueueReceive(scale_queue,scale,0)==pdTRUE) compass_scale(&c,scale[0],scale[1],scale[2]);
        devices_retry(HAL_GetTick()); devices_sample(&c); display_draw(&c);
        for(unsigned part=0;part<32 && display_ready();++part) {
            if(display_chunk()) { ++c.frames; c.frame_ms=HAL_GetTick()-previous; previous=HAL_GetTick(); }
        }
        c.display_ok=display_ready();
        xQueueOverwrite(status_queue,&c);
        xTaskDelayUntil(&wake,pdMS_TO_TICKS(40));
    }
}
/* Only this task calls foreground UART functions; ISR is the queue counterpart. */
static void communications_task(void *arg) {
    (void)arg; Compass latest; compass_init(&latest);
    char input[80]; unsigned used=0; bool overflow=false;
    uint32_t report=HAL_GetTick();
    serial_write("READY COMPASS 1 LR4 FREERTOS\r\n");
    for(;;) {
        serial_service();
        (void)xQueuePeek(status_queue,&latest,0);
        uint8_t b;
        for(unsigned budget=0;budget<32 && serial_tx_space()>180 && serial_read(&b);++budget) {
            if(b=='\r') continue;
            if(b=='\n') {
                input[used]=0; unsigned scale[3]; int cmd=overflow?0:compass_command(input,scale);
                if(cmd==1) serial_write("PONG COMPASS 1\r\n");
                else if(cmd==2) { char reply[180]; compass_status(&latest,reply,sizeof reply);
    serial_write(reply); }
                else if(cmd==3) { xQueueOverwrite(scale_queue,scale); serial_write("OK SCALE QUEUED\r\n"); }
                else serial_write("ERR COMMAND\r\n");
                used=0; overflow=false;
            } else if(used<sizeof input-1) input[used++]=(char)b; else overflow=true;
        }
        if((uint32_t)(HAL_GetTick()-report)>=100 && serial_tx_space()>180) {
            char reply[180]; compass_status(&latest,reply,sizeof reply); serial_write(reply);
    report=HAL_GetTick();
        }
        vTaskDelay(pdMS_TO_TICKS(1));
    }
}
int main(void) {
    board_init(); serial_init(115200);
    scale_queue=xQueueCreate(1,sizeof(unsigned[3])); status_queue=xQueueCreate(1,sizeof(Compass));
    if(!scale_queue||!status_queue) fatal();
    if(xTaskCreate(display_task,"display",1024,0,2,0)!=pdPASS) fatal();
    if(xTaskCreate(communications_task,"uart",768,0,3,0)!=pdPASS) fatal();
    vTaskStartScheduler(); fatal();
}
```

Полные исходные коды всех модулей, тестов и клиента входят в папку лабораторной. Здесь приведён фрагмент реализации.

10.2. Исходные коды: прикладная логика

Compass_RTOS_LR4_V2/Core/Src/compass.c

```
#include "compass.h"
#include <math.h>
#include <stdio.h>
#include <string.h>
void compass_init(Compass *c) { *c=(Compass){.scale={100,100,100},.heading=-1}; }
bool compass_scale(Compass *c,unsigned x,unsigned y,unsigned z) {
    if(x>200||y>200||z>200) return false;
    c->scale[0]=(uint16_t)x; c->scale[1]=(uint16_t)y; c->scale[2]=(uint16_t)z; return true;
}
int compass_heading(int32_t x,int32_t y) {
    if(x==0 && y==0) return -1;
    float angle=atan2f((float)y,(float)x)*5729.5779513f;
    int result=(int)lroundf(angle); if(result<0) result+=36000; return result%36000;
}
void compass_measure(Compass *c,const int16_t raw[3],int sensitivity) {
    for(unsigned i=0;i<3;++i) c->field[i]=(int32_t)((int64_t)raw[i]*sensitivity*c->scale[i]/100000);
    c->heading=compass_heading(c->field[0],c->field[1]);
}
int compass_command(const char *s,unsigned scales[3]) {
    if(!strcmp(s,"PING")) return 1;
    if(!strcmp(s,"GET")) return 2;
    if(strncmp(s,"SCALE ",6)) return 0;
    s+=6;
    for(unsigned i=0;i<3;++i) {
        if(*s<'0'||*s>'9') return 0;
        unsigned value=0;
        do { value=value*10+(unsigned)(*s++-'0'); if(value>200) return 0; } while(*s>='0'&&*s<='9');
        scales[i]=value;
        if(i<2) { if(*s++!=' ') return 0; } else if(*s) return 0;
    }
    return 3;
}
void compass_status(const Compass *c,char *out,unsigned size) {
    snprintf(out,size,"DATA %lu %ld %ld %ld %d %u %u %u %u %u %u %lu %lu\r\n",
       (unsigned long)c->tick,(long)c->field[0],(long)c->field[1],(long)c->field[2],c->heading,
       c->scale[0],c->scale[1],c->scale[2],c->sensor_ok,c->display_ok,c->sensor_id,
       (unsigned long)c->frames,(unsigned long)c->frame_ms);
}
```

Полные исходные коды всех модулей, тестов и клиента входят в папку лабораторной. Здесь приведён фрагмент реализации.

10.3. Исходные коды: драйвер устройств

Compass_RTOS_LR4_V2/Core/Src/devices.c

```
}
static void magnetometer_init(void) {
    mag_id=0; mag_address=0;
    /* SDK boards normally carry LSM9DS1; identify instead of guessing revision. */
    const uint8_t addresses[]={0x1c,0x1e,0x1d};
    for(unsigned i=0;i<sizeof addresses;++i) {
        uint8_t id=0;
        if(!readreg(addresses[i],0x0f,&id,1)) continue;
        if(id==0x3d) {
            if(!writereg(addresses[i],0x20,0x7c)||!writereg(addresses[i],0x21,0x00)||

    !writereg(addresses[i],0x22,0x00)||!writereg(addresses[i],0x23,0x0c)||!writereg(addresses[i],0x24,0x40))
     continue;
            sensitivity=14615;
        } else if(id==0x49) {
            if(!writereg(addresses[i],0x20,0x08)||!writereg(addresses[i],0x24,0x74)||
               !writereg(addresses[i],0x25,0)||!writereg(addresses[i],0x26,0)) continue;
            sensitivity=8000;
        } else continue;
        mag_address=addresses[i]; mag_id=id; last_sample=HAL_GetTick(); return;
    }
}
static void oled_init(void) {
    oled_ok=false; chunk=0;
    uint8_t config,output;
    if(!readreg(0x70,3,&config,1)||!readreg(0x70,1,&output,1)) return;
    /* Preserve other expander pins; P0 alone controls OLED reset. */
    if(!writereg(0x70,1,output|1)||!writereg(0x70,3,config&0xfe)) return;
    HAL_Delay(1);
    if(!writereg(0x70,1,output&0xfe)) return;
    HAL_Delay(1);
    if(!writereg(0x70,1,output|1)) return;
    HAL_Delay(2);
    const uint8_t init[]={0xae,0x20,0x00,0xc8,0x40,0x81,0xff,0xa1,0xa6,0xa8,0x3f,0xa4,0xd3,0,0xd5,0xf0,0xd9,
    0x22,0xda,0x12,0xdb,0x20,0x8d,0x14,0xaf};
    oled_ok=oled_command(init,sizeof init);
}
void devices_init(void) {
    __HAL_RCC_GPIOB_CLK_ENABLE(); __HAL_RCC_I2C1_CLK_ENABLE();
    GPIO_InitTypeDef io={0}; io.Pin=GPIO_PIN_8|GPIO_PIN_9; io.Mode=GPIO_MODE_AF_OD;
    io.Pull=GPIO_NOPULL; io.Speed=GPIO_SPEED_FREQ_VERY_HIGH; io.Alternate=GPIO_AF4_I2C1;
    HAL_GPIO_Init(GPIOB,&io);
    bus.Instance=I2C1; bus.Init.ClockSpeed=400000; bus.Init.DutyCycle=I2C_DUTYCYCLE_2;
    bus.Init.AddressingMode=I2C_ADDRESSINGMODE_7BIT;
    if(HAL_I2C_Init(&bus)!=HAL_OK) fatal();
    magnetometer_init(); oled_init(); last_retry=HAL_GetTick();
}
void devices_retry(uint32_t now) {
    if((uint32_t)(now-last_retry)<1000) return;
    last_retry=now;
    if(!mag_address) magnetometer_init();
    if(!oled_ok) oled_init();
}
void devices_sample(Compass *c) {
    c->tick=HAL_GetTick(); c->sensor_id=mag_id; c->display_ok=oled_ok;
```

Полные исходные коды всех модулей, тестов и клиента входят в папку лабораторной. Здесь приведён фрагмент реализации.

11. Проверка на стенде, выводы и источники

План испытаний

• Подтвердить PONG, sensor_ok=1 и oled_ok=1, сравнить компоненты OLED/ПК.
• Проверить масштабы 0/100/200%, все нули и поворот платы на столе.
• Сопоставить реальные оси с направлением стрелки, проверить локальный магнит.
• Наблюдать F:99→00 и измерить delta_frames/delta_tick за ≥10 с, приложить UART-лог.

Выводы

Подготовлены компас, драйверы I²C/OLED/магнитометра и клиент с журналом. Выполнен перенос на две задачи FreeRTOS с очередями снимков и независимым HAL tick. Работа физических интерфейсов и аппаратные характеристики пока не подтверждены. Протокол наблюдений вынесен в ПРОВЕРКА_НА_ПЛАТЕ.md.

Источники

Учебное пособие SDK-1.1M_26101101.pdf, разделы 1.3–1.8, 5.2–5.4.
STMicroelectronics RM0090:
www.st.com/resource/en/reference_manual/rm0090-stm32f407-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
HAL STM32F4: github.com/STMicroelectronics/stm32f4xx-hal-driver
LSM9DS1/LSM9DS0: www.st.com/resource/en/datasheet/lsm9ds1.pdf; lsm9ds0.pdf.
Аппаратная платформа: github.com/lmtspbru/SDK-1.1M.
FreeRTOS Kernel V11.2.0: github.com/FreeRTOS/FreeRTOS-Kernel/tree/V11.2.0.